

```

#!/usr/bin/env python3

from flask import Flask, Response
from picamera2 import Picamera2
import cv2
import time
import serial
import threading
import math

# ----- SETTINGS -----
STOP_DISTANCE_M = 0.50

M1_PWM = 1400
M2_PWM = 1400
M3_PWM = 1425
M4_PWM = 1425

app = Flask(__name__)

latest_jpeg = None
frame_lock = threading.Lock()

person_detected = False
front_distance = None
robot_status = "Starting..."

# ----- CAMERA -----
picam2 = Picamera2()
config = picam2.create_video_configuration(
    main={"size": (640, 480), "format": "RGB888"}
)
picam2.configure(config)
picam2.start()
time.sleep(2)

hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())

# ----- MOTOR -----
ser = serial.Serial('/dev/stm32', 115200, timeout=1)
time.sleep(0.2)

def send(cmd):

```

```

ser.write((cmd + '\n').encode())
print(cmd)

def setup_motors():
    for cmd in [
        '$mtype:1#',
        '$mphase:30#',
        '$mline:11#',
        '$wdiameter:97#',
        '$deadzone:1400#':
    ]:
        send(cmd)
        time.sleep(0.1)

def move_forward():
    send(f'$pwm:{M1_PWM},{M2_PWM},{M3_PWM},{M4_PWM}#')

def stop_motors():
    send('$pwm:0,0,0,0#')

# ----- LIDAR -----
class Lidar:
    def __init__(self):
        self._lock = threading.Lock()
        self._points = []
        self._running = False
        self._laser = None
        self._yd = None

    import ydlidar as _yd
    self._yd = _yd
    _yd.os_init()

    laser = _yd.CYdLidar()
    laser.setlidaropt(_yd.LidarPropSerialPort, '/dev/ydlidar')
    laser.setlidaropt(_yd.LidarPropSerialBaudrate, 230400)
    laser.setlidaropt(_yd.LidarPropLidarType, _yd.TYPE_TRIANGLE)
    laser.setlidaropt(_yd.LidarPropDeviceType, _yd.YDLIDAR_TYPE_SERIAL)
    laser.setlidaropt(_yd.LidarPropScanFrequency, 10.0)
    laser.setlidaropt(_yd.LidarPropSampleRate, 4)
    laser.setlidaropt(_yd.LidarPropSingleChannel, False)
    laser.setlidaropt(_yd.LidarPropIntenstiy, True)
    laser.setlidaropt(_yd.LidarPropMaxAngle, 180.0)
    laser.setlidaropt(_yd.LidarPropMinAngle, -180.0)

```

```

if not laser.initialize():
    print("LiDAR failed to initialize")
    return

if not laser.turnOn():
    print("LiDAR failed to turn on")
    return

self._laser = laser
self._running = True
threading.Thread(target=self._loop, daemon=True).start()
print("LiDAR started")

def _loop(self):
    scan = self._yd.LaserScan()

    while self._running:
        try:
            if self._laser.doProcessSimple(scan):
                pts = [
                    (p.angle, p.range)
                    for p in scan.points
                    if 0.02 < p.range < 16.0
                ]
                with self._lock:
                    self._points = pts
            except Exception as e:
                print("LiDAR error:", e)
                break

def get_scan(self):
    with self._lock:
        return list(self._points)

def close(self):
    self._running = False
    if self._laser:
        self._laser.turnOff()
        self._laser.disconnecting()

def get_front_distance(lidar):
    points = lidar.get_scan()

```

```

if not points:
    return None

distances = []

for angle_rad, distance_m in points:
    angle_deg = math.degrees(angle_rad)

    # Calibrated front 180 degrees
    if angle_deg >= 90 or angle_deg <= -90:
        distances.append(distance_m)

if not distances:
    return None

return min(distances)

# ----- CAMERA LOOP -----
def camera_loop():
    global latest_jpeg, person_detected, robot_status, front_distance

    while True:
        frame = picam2.capture_array()

        boxes, weights = hog.detectMultiScale(
            frame,
            winStride=(8, 8),
            padding=(8, 8),
            scale=1.05
        )

        person_detected = len(boxes) > 0

        for (x, y, w, h) in boxes:
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
            cv2.putText(frame, "Person", (x, y-10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

            cv2.putText(frame, f"Person: {person_detected}", (20, 35),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8,
                (0,255,0) if person_detected else (0,0,255), 2)

        if front_distance is not None:
            cv2.putText(frame, f"Front Distance: {front_distance:.2f} m", (20, 70),

```

```

        cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255,255,255), 2)
    else:
        cv2.putText(frame, "Front Distance: None", (20, 70),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255,255,255), 2)

    cv2.putText(frame, robot_status, (20, 105),
        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255,255,255), 2)

    ret, buffer = cv2.imencode(".jpg", frame)

    if ret:
        with frame_lock:
            latest_jpeg = buffer.tobytes()

    time.sleep(0.05)

# ----- ROBOT LOOP -----
def robot_loop():
    global front_distance, robot_status

    setup_motors()
    lidar = Lidar()
    time.sleep(2)

    try:
        while True:
            front_distance = get_front_distance(lidar)

            print("-----")
            print("Person detected:", person_detected)
            print("Front distance:", front_distance)

            if person_detected:
                if front_distance is not None and front_distance <= STOP_DISTANCE_M:
                    robot_status = "STOP - within 50 cm"
                    stop_motors()
                else:
                    robot_status = "APPROACHING PERSON"
                    move_forward()
            else:
                robot_status = "STOP - no person"
                stop_motors()

            print("Status:", robot_status)

```

```

        time.sleep(0.2)

    finally:
        stop_motors()
        lidar.close()
        ser.close()

# ----- WEB STREAM -----
def generate_frames():
    while True:
        with frame_lock:
            frame = latest_jpeg

        if frame is None:
            time.sleep(0.1)
            continue

        yield (
            b"--frame\r\n"
            b"Content-Type: image/jpeg\r\n\r\n" +
            frame +
            b"\r\n"
        )

        time.sleep(0.05)

@app.route("/")
def index():
    return """
<html>
  <head><title>LifeLineBot</title></head>
  <body style="text-align:center; font-family:Arial;">
    <h1>LifeLineBot Person Approach Test</h1>
    
    <p>Camera detects person. LiDAR stops robot at 50 cm.</p>
  </body>
</html>
"""

@app.route("/video_feed")
def video_feed():

```